R-Übungsaufgaben zu MfW1 - Mengenlehre und Algebra

Im Rahmen der R-Übungsaufgaben haben Sie die Möglichkeit sich mit der Software R praktisch vertraut zu machen. Wir empfehlen hierbei, sich intenstiv mit den Aufgaben zu beschäftigen. D.h. versuchen Sie die Code-Bestandteile eigenständig zu erklären und nehmen Sie eigenständig Variationen dieser vor, um die Wirkungsweise nachvollziehen zu können.
Die R-Aufgaben erheben keinen Anspruch auf Vollständigkeit, d.h. nur weil nicht alle in der Vorlesung behandelten Themen auf diesem Übungszettel vorkommen, heißt dies nicht, dass diese nicht klausurrelevant sind!

Hinweis: Beachten Sie, dass sämtliche Aufgaben auf den R Übungsblättern auch via R gelöst werden müssen, sofern nicht anders gekennzeichnet! Beachten Sie ferner, dass die Aufgabenmenge keinen Hinweis auf die Relevanz des Themas darstellt.

R 1.1 - Grundlagen in R

In diesem Kapitel werden Sie mit den allgemeinen Grundlagen von der Software R konfrontiert.

Aufgabe 1.

(a) Öffnen Sie ein neues Skript in R. Hinweis: Arbeiten Sie grundsätzlich im Skript, statt über die Console.

(b) Um Erweiterungen bzgl. ergänzende Funktionen auch in R umsetzen zu können, ist es notwendig, die entsprechenden R-Pakete zu installieren. Tun Sie dies beispielsweise für das Paket `numDeriv`. Laden Sie anschließend das Paket mit dem entsprechendem Befehl.

(c) Da R eine englischsprachige Programmiersprache ist, werden sog. floats (Punkte) statt Kommata als Dezimaltrennzeichen verwendet. Machen Sie sich mit dem Unterschied vertraut, indem Sie einmal 5.9 und $5,9$ in R eingeben und ausführen.

(d) Bei Fragen bzw. Unklarheiten zu Methoden oder zu der Interpretation der Ergebnisse gibt es die Möglichkeit in R, auf englischsprachige Hilfe zurückzugreifen. Benutzen Sie den entsprechenden Befehl, um sich mit der Hilfe zur Quadratwurzel auseinanderzusetzen.
Hinweis: Es sind die entsprechenden englischsprachigen Begriffe in der Suche einzugeben!

Kontrollergebnis

(a) Ein neues Skript kann entsprechend über die Reiter "File" -> "New File" -> "R Script" geöffnet werden.
(b)

```
install.packages("numDeriv")
library(numDeriv)
```

(c)

```
> 5.9
[1] 5.9
> 5,9
Error: unexpected ',' in "5,"
```

(d)

```
?sqrt
help(sqrt)
```

Aufgabe 2.
Bei dieser Aufgabe sollen Sie sich allgemein mit dem Thema Objekte in R beschäftigen.

(a) Sie können in R sowohl Zahlen als auch Strings abspeichern. Setzen Sie sich mit den Unterschieden exemplarisch auseinander, indem Sie in dem Objekt zahl1 die Zahl 1 und in dem Objekt z2 die Zahl 2 abspeichern.
Ferner speichern Sie in dem Objekt b1 den Buchstaben "b" ab.
Versuchen Sie die Ihnen bekannte mathematischen Operationen mit den gepeicherten Objekten durchzuführen. Was fällt Ihnen auf?
Was passiert, wenn Sie die Zahlen 1 und 2 in den Objekten z3 und z4 jeweils als String abspeichern und die mathematischen Operationen ausführen?

(b) Sie können mehrere Objekte in einem zusammenfassen, sodass ein Vektorobjekt entsteht.
Setzen Sie dies mit dem entsprechenden Befehl in R exemplarisch um, indem Sie im Objekt d die Zahlen $1, 2, 3$ und 4 abspeichern. Überdies speichern Sie in dem Objekt n die Namen Hannah, Eva, Paul sowie Peter ab.

(c) Des Weiteren haben Sie über R die Möglichkeit, bestimmte Elemente innerhalb eines Objektes aufzurufen. Rufen Sie mit einem Befehl die Zahl 4 im Objekt d auf. Rufen Sie zudem den Namen Paul in dem Objekt n.

(d) Definieren Sie das Objekt e in R mit den Zahlen $2, 3, 4$ und 5.
Sie können Objekte miteinander verrechnen, sofern möglich. Nutzen Sie einige Ihnen bekannte Rechenmethoden, um die Objekte n, d sowie e miteinander zu verrechnen.

Kontrollergebnis
(a)

```
> zahl1 <- 1
> z2 <- 2
> b1 <- "b" #Ein String wird in Anfuehrungszeichen gesetzt.
> zahl1+z2
[1] 3
> zahl1+b1
Error in zahl1 + b1 : non-numeric argument to binary operator
> zahl1
[1] 1
> z3 <- "3"
> z4 <- "4"
> z3
[1] "3"
```

(b)

```
> d <- c(1,2,3,4)
> n <- c("Hannah", "Eva", "Paul", "Peter")
> d
[1] 1 2 3 4
> n
[1] "Hannah" "Eva"    "Paul"   "Peter"
```

(c)

```
> d[4]
[1] 4
> n[3]
[1] "Paul"
```

(d)

```
> e <- c(2,3,4,5)
> # Hier einige exemplarische Rechnungen
> n+d
Error in n + d : non-numeric argument to binary operator
> n+e
Error in n + e : non-numeric argument to binary operator
> n+n
Error in n + n : non-numeric argument to binary operator
> # Strings k nnen nicht miteinander verrechnet werden.
> d*e
[1]  2  6 12 20
> 2d
Error: unexpected symbol in "2d"
> # Fehlermeldung, da * fehlt
> 2*d
[1] 2 4 6 8
> d-e
[1] -1 -1 -1 -1
> e^2
[1]  4  9 16 25
> d/e
```

```
22 [1] 0.5000000 0.6666667 0.7500000 0.8000000
```

Aufgabe 3.
Bei dieser Aufgabe sollen Sie sich allgemein mit dem Thema Sequenzen in R beschäftigen.

(a) Sie können in R Sequenzen bzw. Zahlenfolgen definieren, um so effizienter in R arbeiten zu können. Definieren Sie eine exemplarische Sequenz ausgehend von 1 bis 20 in dem Objekt a.

(b) Definieren Sie überdies eine Sequenz mit vordefinierten Start- und Endwerten in dem Objekt b. Wählen Sie exemplarisch als Startwert 10 und als Endwert 4 aus.

(c) Des Weiteren definieren Sie in dem Objekt c eine Sequenz von -8 bis 16, wobei dieser Sequenz eine Inkrementierung von 4 zugrunde gelegt wird.

(d) Geben Sie in dem Objekt d eine Sequenz von 0 bis 19 an, wobei diese eine vordefinierte Länge von 5 umfassen soll.

Kontrollergebnis
(a)

```
1 > a <- 1:20
2 > a <- seq(20)
3 > a <- seq(1,20,1)
4 > a <- seq(1,20)
5 > a <- seq(from=1, to=20)
6 > a
7  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
```

(b)

```
1 > b <- 10:4
2 > b <- seq(from=10, to=4)
3 > b <- seq(10,4,1)
4 Error in seq.default(10, 4, 1) : falsches Vorzeichen im 'by' Argument
5 > b <- seq(10,4, -1)
6 > b
7 [1] 10  9  8  7  6  5  4
```

(c)

```
1 > c <- seq(from=-8, to=16, by=4)
2 > c <- seq(-8,16,4)
3 > c
4 [1] -8 -4  0  4  8 12 16
```

(d)

```
1 > d <- seq(from=0, to=19, length.out=5)
2 > d
3 [1]  0.00  4.75  9.50 14.25 19.00
```

R 1.2 - Grundrechenarten in R

In diesem Kapitel werden Sie sich mit den grundlegendsten Rechenoperationen in R vertraut machen.

Aufgabe 4.
Berechnen Sie daher für die Variablen $a = 21$ und $b = 2$ folgende Ausdrücke in R unter Verwendung der Variablen:

(a) $a + b$

(b) $a - b$

(c) $a \cdot b$

(d) $\frac{a}{b}$

Kontrollergebnis
Allgemeine numerische Definition der Variablen:

```
a <- 21
b <- 2
```

(a)

```
> a+b
[1] 23
```

(b)

```
> a-b
[1] 19
```

(c)

```
> a*b
[1] 42
```

(d)

```
> a/b
[1] 10.5
# Output mit Punkt fuer Dezimalzahlen!
```

Aufgabe 5.
Geben Sie mit den Variablen $a = 10,5$ und $b = 8$ die folgenden Ausrücke in R an:

(a) $3a + b$

(b) $b + 3a$

(c) Ist das Ergebnis gleich? Begründen Sie.

Kontrollergebnis
Allgemeine numerische Definition der Variablen:

```
a <- 10.5
b <- 8
```

(a)

```
> 3*a+b
[1] 39.5
```

(b)

```
> b+3*a
[1] 39.5
```

(c) Gleiches Ergebnis, da auch in R Punkt- vor Strichrechnung umgesetzt wird

Aufgabe 6.

Berechnen Sie den Ausdruck $\frac{2a}{b-7,5}$ in R. Die Variablen seien wie folgt definiert: $a = 10.5$ und $b = 8$.

Kontrollergebnis

Allgemeine numerische Definition der Variablen:

```
a <- 10.5
b <- 8
```

```
> 2*a/(b-7.5)
[1] 42
# Beachten Sie die Setzung von Klammern!
# Siehe:
> 2*a/b-7.5
[1] -4.875
```

Aufgabe 7.

Berechnen Sie - unter der Annahme, dass die Variablen a und b wie folgt definiert sind: $a = 10.5$ und $b = 10$ - folgende Ausdrücke zunächst händisch und geben Sie ferner die zugehörigen Ausdrücke in R an:

(a) $a - \frac{2,5}{b}$

(b) $\frac{a-2,5}{b}$

Kontrollergebnis

Allgemeine numerische Definition der Variablen:

```
a <- 10.5
b <- 10
```

(a)

```
> a-(2.5/b)
[1] 10.25
```

(b)

```
> (a-2.5)/b
[1] 0.8
```

R 1.3 - Logik in R

Aufgabe 8.
Definieren Sie zunächst in R die beiden Variablen $a = 10$ und $b = 5$.
Führen Sie im Anschluss folgende Vergleiche durch und interpretieren Sie Ihr Ergebnis:

(a) $a < b$

(b) $a \leq b$

(c) $b \geq b$

(d) $b > b$

(e) $a = a$

Kontrollergebnis

```
> # Allgemeine Definition der Variablen:
> a <- 10
> b <- 5
```

(a)

```
> a < b
[1] FALSE
```

(b)

```
> a <= b
[1] FALSE
```

(c)

```
> b >= b
[1] TRUE
```

(d)

```
> b > b
[1] FALSE
```

(e)

```
> a == a # Unterschied zu a = a beachten!
[1] TRUE
```

Aufgabe 9.
Gegeben sei der R Code `a = b` und `a == b`.
Erläutern Sie den Unterschied.

Kontrollergebnis
`a = b` weist der Variable a den Wert b zu und ist hier äquivalent zu `a <- b`.
Mit `a == b` wird geprüft, ob der Wert von a dem Wert von b entspricht. Output dieser Methode kann entweder `TRUE` bzw. `FALSE` sein.

Aufgabe 10.
Führen Sie für $a = 10$ und $b = 5$ folgenden R-Code: `(a<=b)==FALSE` aus.
Obwohl *a* und *b* Zahlen sind und mit einem Wahrheitswert verglichen werden, gibt der Befehl keine Fehlermeldung sondern `TRUE` zurück. Wie können Sie das Ergebnis dieses Befehls erklären?

Kontrollergebnis

```
> a <- 10
> b <- 5
>
> (a<=b)==FALSE
[1] TRUE
```

Der Befehl überprüft, ob $a \leq b$ eine falsche Aussage ist. Da a größer als b ist, ist die Aussage `a <= b` falsch, wodurch die Aussage `(a<=b)==FALSE` wahr ist.